



**CourseCube®**  
(Formerly Java Learning Center)

# OAuth2

## Study Guide

**Author**  
**Srinivas Dande**





**CourseCube<sup>®</sup>**  
( Formerly Java Learning Center )



## Introduction to OAuth2

- OAuth2 is basically a standard **for authorization, not for authentication**.
- It's important to distinguish between these concepts to understand OAuth.
- So, in very simple terms:

**Authentication - Identifying a user**

**Authorization - Determining what the user can access**

- OAuth allows applications to obtain limited access to a service or to a resource.
- One of the core goals of OAuth is to help users to no longer have to provide their credentials directly to third-party applications to allow access to their resources.
- Four main components of the OAuth 2.0 specification:
  - 1) **Resource Owner**
  - 2) **Client Application**
  - 3) **Authorization Server**
  - 4) **Resource Server**
- OAuth2 enables third-party application (such as **JLCBookStore**) to automatically login to third-party application (**OAuth Providers**)
- Following are the third Party **OAuth Providers**:
  - Twitter
  - Facebook
  - LinkedIn
  - Google
  - GitHub
- Through OAuth2 login, our application obtains limited access on Twitter or Facebook or LinkedIn or Google or GitHub resources than direct login to social.
- Microservices architecture says decompose Single Application into many smaller services. But we cannot ask every end user to sign-up and sign-in to every microservice. Instead the end user prefers Single Sign-On irrespective of number of microservices.
- In case of Single Sign-On, the common id is required to represent user's identity.



- Different microservices will run on different embedded servers. Hence there is no common session id. That means we cannot use session id across different microservices to represent user's identity.
- Instead of using session id, we can use oauth token which is common across all microservices to represent user's identity.
- The oauth token was generated by Authorization server and validated by Resource server.
- Both Authorization server and Resource server together called as OAuth provider.
- The microservices projects can use either existing OAuth providers (such as Twitter or Facebook or LinkedIn or Google or GitHub) OR implement our own OAuth Provider.
- **The Authorization server generates access token only if resource owner successfully authenticated and authorized.**
- **The Authorization Server issues access token to client application only if client application successfully authenticated.**
- **The Resource Server validates access token to protect resources.**
- We can separate Authorization Server and Resource server.
- In microservice projects there are multiple micro services. Each micro service contains a few resources. These resources need to be protected by Resource server. Hence every microservice needs resource server to protect resources by validating the token i.e., multiple resource servers are needed.
- But only one authorization server is sufficient to issue a common token for all microservices.
- **So implement one authorization server independently and include a separate resource server in each microservice.**



### **Client Registration**

- Client Application (such as Redbus/**JLCBookStore**) should be registered with the OAuth provider (such as facebook).
- The client registration is needed by OAuth provider in order to know whom (client application) to issue access token.
- Also, the client registration is needed by client application in-order to know which resources can be allowed to access by OAuth provider.

### **Client Identifier (Client Id):**

- After client registration, a client identifier is issued to client application

**Ex:**

App ID: 596210484124781 (in case of Facebook)

jlcapp (in our Examples)

### **Client Secret:**

- After client registration, a client secret is issued to client application.

**Ex:**

App Secret: 126804b952a0ad7d193e8d6a2b982632 (in case of Facebook)

jlc123 (in our Examples)

### **Client Authentication**

- The client application (such as Redbus/**JLCBookStore**) should be authenticated itself with OAuth provider (such as facebook) with the help of App ID and App Secret. This process is called as client authentication.
- The client authentication is needed to get OAuth token from OAuth provider by providing client\_id and client\_secret

### **Protocol Endpoints**

The authorization process utilizes two authorization server endpoints:

- 1) Authorization endpoint (/oauth/authorize)
- 2) Token endpoint (/oauth/token)



### **Authorization Endpoint:**

- The authorization endpoint (/oauth/authorize) is used by the client application to obtain authorization from the resource owner.
- Resource owner authentication is required to access this end point.
- Following is the authorization request URL:

**`http://localhost:12345/oauth/authorize?client_id=jlcapp&redirect_uri=http://localhost:5000/callback&response_type=code&scope=read_profile`**

- The above URL will prompt login pop-up to enter resource owner's username and password because accessing of authorization endpoint (/oauth/authorize) needs resource owner authentication.
- In case of microservice projects, we can define resource owner's credentials in application.properties file across all microservices.

```
security.user.name=srinivas  
security.user.password=srinivas
```

- After authentication, we will get authorization page as below for the Resource Owner Approval.

### **Token Endpoint**

- The token endpoint (/oauth/token) is used by the client application to generate and obtain an access token from the authorization server.
- Client authentication is required to access this end point.
- Following is the access token request URL:

**`curl -X POST http://localhost:12345/oauth/token --user jlcapp:jlc123 -H "content-type: application/x-www-form-urlencoded" -d "code=N7hSH3&grant_type=authorization_code&redirect_uri=http://localhost:5000/callback&scope=read_profile"`**

- Access token acts as a credential to access oauth protected resources.